



Programação Orientada a Objetos

Trabalho Prático 22/23

Simulador de uma Reserva Natural

Tiago Rafael Santos Cardoso - 2021138999

Daniela Fernandes Correia – 2021143404

Índice

Introdução	3
Classes	3
Classe Simulador	3
Classe Reserva	5
Classe Animal	6
Classe Alimento	7
Classe Comandos.....	8
Conclusão	10

Introdução

No âmbito da unidade curricular de Programação Orientada a Objetos foi-nos proposto o desenvolvimento, em C++, de um simulador de uma reserva natural povoada por diversos animais.

Queremos desde já referir que a nossa resolução do trabalho foi desenvolvida no standard C++14.

Classes desenvolvidas:

1. Classe Simulador:

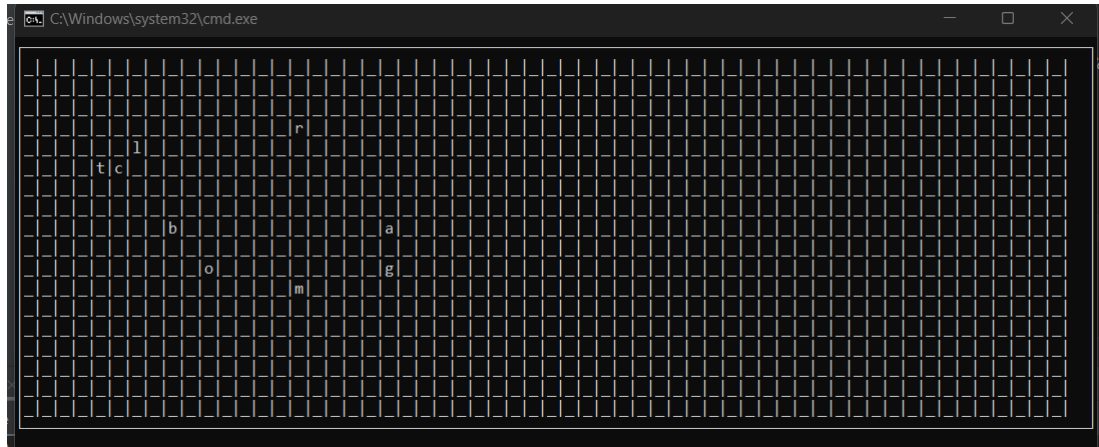
```
#ifndef TRABALHO_1__SIMULADOR_H
#define TRABALHO_1__SIMULADOR_H
#include ...
using namespace std;
using namespace term;
class Simulador {
public:
    Simulador(Reserva &r,int wr,int hr);
    ~Simulador();
    int introduzComandos(int &xr,int &yr);
    int validaComandos(string &comando,int& xr, int &yr,string &output,Window &zonaR,Window &zona0);
    void ficheiroConstantes();
    void iniciaSimulacao();
    void mostraOutputVisivel(string &output,Window &zona0,int y);
    void helper(string&output);
private:
    Reserva r;
    vector<Reserva*> estados;
    int wr,hr;
};
```

Esta classe está responsável por guardar a reserva gerada e os vários “estados” que pretendem ser guardados.

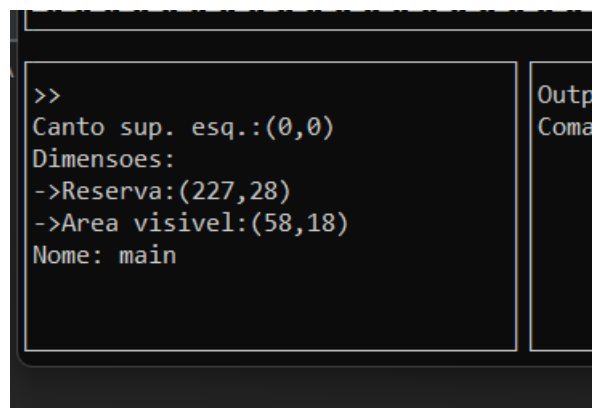
Esta classe é também responsável por gerar, e fazer a manutenção do conteúdo das várias zonas (todas com tamanho estático) que aparecem no ecrã, para desta forma o utilizador estar a par do que está a acontecer na reserva.

As zonas referidas acima são:

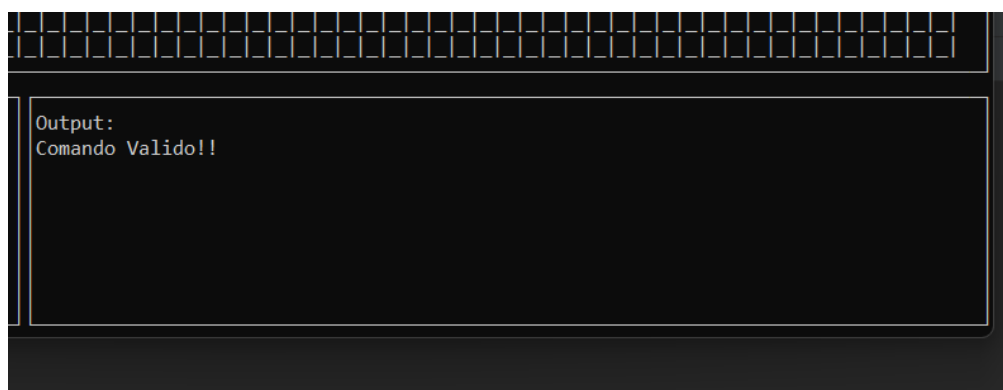
zonaR -> Onde é mostrada o que for possível da reserva e observar em que localização da reserva o utilizador se encontra.



zonaC -> Onde é possível introduzir os comandos. Mostra informações necessárias para o conhecimento do utilizador.



zonaO -> Onde é mostrado o output de cada comando que é introduzido na zonaC assim como a sua validez.



Nesta zona caso o output seja extenso para o tamanho da janela optamos por acrescentar a possibilidade de navegar somente nessa janela com o comando CTRL+KEY_UP (para cima) ou CTRL+KEY_DOWN (para baixo). Isto foi possível acrescentando no ficheiro Terminal.cpp a leitura dessa combinação de teclas.

2. Classe Reserva:

```
using namespace term;
class Reserva {
public:
    Reserva(const int NL,const int NC,string nome);
    Reserva(Reserva &o);
    ~Reserva();
    void insereAnimal(string especie,int x=-1,int y=-1);
    void insereFilhote(Animal* a);
    void insereAlimento(string nomeAlimento, int vn,int tx,int x = -1,int y = -1);
    void insereAlimento(Alimento *a);
    string mostraInfoAnimais();
    string mostraAnimaisVisiveis(int wr, int hr,int xr,int yr);
    string findTipo(int x,int y)const;
    void mostraReservaVisivel(int xr, int yr,int wr,int hr,Window &zonaR) const;
    void eliminaAnimal(int x, int y=-1);
    void eliminaAlimento(int x,int y=-1);
    void foiComido(int idd);
    string getRedor(int xmin,int xmax,int ymin,int ymax,int idd,string cheiro)const;
    const string getNome()const;
    string getInfoById(int num);
    string getInfoByPosition(int x,int y);
    const int getLinhas()const;
    const int getColunas()const;
    int setNewId();
    void setName(string n);
    string feed(int ptn,int ptt,int x,int y=-1);
    string n();
    void DeuAVolta(int &i,int &j)const;
    Reserva &operator=(const Reserva&b);
private:
    int id = 0;
    string nome;
    const int NL;
    const int NC;
    vector<Animal*> animais;
    vector<Animal*> filhotes;
    vector<Animal*> mortos;
    vector<Alimento*> alimento;
    vector<Alimento*> novoAlimento;
    vector<Alimento*> alimentoMorto;
};
```

Nesta classe são guardados os animais e alimentos presentes na reserva, a dimensão da reserva, a quantidade de alimentos e animais já introduzidos.

As funcionalidades aplicadas aqui servem para principalmente auxiliar a execução dos comandos e responderem a pedidos feitos pelo utilizador, como obter informação dos animais visíveis, inserir animais e alimentos.

Aqui também é importante referir que se gere tudo o que é animais mortos e animais que nascem, o mesmo com os alimentos, eliminando tudo o que estiver morto e inserindo o que tiver nascido.

3. Classe Animal:

```
using namespace std;
class Reserva;
class Animal {
public:
    Animal(string especie, string tipo, int peso, int x, int y, int identificador, Reserva &r, int saudePai=-1);
    virtual string getAsString() const=0;
    int getId()const;
    int getX()const;
    int getY()const;
    string getTipo()const;
    int getPeso()const;
    virtual Animal* duplica()=0;
    void comer(int ptNutricao, int ptToxicidade, string tipodecomida, int idAlimento);
    string getHist();
    virtual void setRedor()=0;
    virtual string move()=0;
    void RandomSentido();
    void Segue(int x1, int y1);
    void Foge(int x1, int y1);
    bool IsDead();
    int getConstantes(string e);

protected:
    int x, y, raioMovimento;
    string especie, tipo;
    int identificador, tempoDeVida;
    int peso;
    int saude;
    int VAnimal;
    int fome;
    int tempoParalizado;
    bool died;
    bool freeze;
    Reserva &r;
    vector<string> redor;
private:
    vector<HistoricoAlimento> hist;
};
```

Esta classe é a base para os animais existentes na reserva, aqui foram aplicados os métodos que são comuns a todos os animais, como, deslocar-se (*void RandomSentido(...)*, *void Segue(...)*, *void Foge(...)*) e comer.

Os animais que derivam desta base são:

Canguru, Coelho, Lobo, Ovelha, Mistério.

O **animal Mistério** possui as seguintes características:

- Apenas consegue ver o que está uma posição à sua volta.
- Apenas consome alimentos que cheirem a verdura, mas estes não servem para se alimentar, servem sim para se reproduzir.
- Quando se reproduz o filho aparece aleatoriamente num raio de 2 posições.
- Não passa fome.
- Não perde pontos de saúde.
- Morre espontaneamente passado 100 iterações.

Cada animal aplica depois os métodos declarados aqui virtuais, tais como, obter o que esta ao seu redor, duplicar, e mover-se consoante o meio que o rodeia *string move(...)*.

4. Classe Alimento:

```
using namespace std;
class Reserva;
class Alimento {
public:
    Alimento(string nome, string tipo, int vn, int tx, int x, int y, int identificador, initializer_list<string> cheiros, Reserva &r);
    string getAsString() const;
    int getId() const;
    int getX() const;
    int getY() const;
    string getTipo() const;
    virtual Alimento* duplica()=0;
    bool smellLike(string cheiro) const;
    string getIntData();
    virtual void move()=0;
    bool isDead();
    int getConstantes(string e);
protected:
    int x, y;
    string nome, tipo;
    int VAlimento;
    int valorNutritivo;
    int toxicidade;
    int identificador, tempoDeVida;
    bool died;
    Reserva &r;
    vector<string> cheiros;
};
```

A classe Alimento também, como nos animais, é a base para os alimentos existentes na reserva.

Os alimentos que derivam desta base são:

Bife, Cenoura, Corpo, Relva, Alimento Mistério.

O **alimento Mistério** possui as seguintes características:

- Este alimento agrada qualquer animal, cheira ao que o animal que passe por ali procura para comer.
- Não é tóxico nem tem pontos de nutrição.
- Ao ser consumido o animal paralisa durante 5 iterações.

Os alimentos são mais simples, não se movem nem comem, aqui não há muito que seja importante referir, cada alimento é responsável com a sua parte que foi requerida no enunciado.

5. Classe Comandos:

```
using namespace std;

class Comandos {
public:
    Comandos(string comando);
    virtual string executa(Reserva &r)=0;
protected:
    string comando;
};
```

Aqui apenas é declarada como virtual o método que cada derivada desta base tem de executar para fazer cumprir o que foi requerido.

Os comandos que derivam desta base são:

Animal (comando animal), *Kill*, *Food*, *Feed*, *NoFood*, *Empty*, *See*, *Info*, *N*, *Anim*, *Visanim*, *Store*, *Restore*, *Load*, *Slide*, *Exit*.

Neste trabalho, aplicamos um método para gerar comandos, alimentos e animais conhecido por *factory method*, que é um método de gerar objetos sem especificar concretamente o seu tipo.

Isto foi aplicado nas classes *FabricaAlimentos*, *FabricaAnimais* e *FabricaComando*, o que facilitou muito no desenvolvimento deste projeto.

1 FabricaAlimentos

```
#include "FabricaAlimentos.h"
FabricaAlimentos::FabricaAlimentos(string nomeAlimento, int x, int y, int id, int vn, int tx): nomeAlimento(nomeAlimento), x(x), y(y), id(id), vn(vn), tx(tx){}

Alimento *FabricaAlimentos::CriaAlimento(Reserva&r) {
    Alimento *a= nullptr;
    if(nomeAlimento=="r"){
        a=new Relva(x,y, id, &r);
    }else if(nomeAlimento=="t"){
        a=new Cenoura(x,y, id, &r);
    }else if(nomeAlimento=="b"){
        a=new Bife(x,y, id, &r);
    }else if(nomeAlimento=="a"){
        a=new AlimentoMisterio(x,y, id, &r);
    }
    return a;
}
```


3. FabricaAnimais

```
#include "FabricaAnimais.h"
FabricaAnimais::FabricaAnimais(string especie,int x,int y, int id):especie(especie),x(x),y(y),id(id){}

Animal *FabricaAnimais::CriaAnimal(Reserva &r,int idPai) {
    if(especie == "c"){
        return new Coelho(x,y, id, &r);
    }else if(especie == "l"){
        return new Lobo(x,y, id, &r);
    }else if(especie == "o"){
        return new Ovelha(x,y, id, &r);
    }else if(especie == "g"){
        return new Canguro(x,y, id,idPai, &r);
    }else if(especie == "m"){
        return new Misterio(x,y, id, &r);
    }
    return nullptr;
}
```

2. FabricaComandos

```
FabricaComandos::FabricaComandos(string comando,int wr,int hr):comando(comando),wr(wr),hr(hr) {}

Comandos *FabricaComandos::CriaComando(int &xr,int &yr,vector<Reserva*>&estados,Window &zonaR,Window &zona0) {
    Comandos *b= nullptr;
    string com;
    istringstream linha( str comando);
    linha>>com;
    if(com=="animal"){
        b=new CAnimal(comando);
    }else if(com=="kill"||com=="killid"){
        b=new Kill(comando);
    }else if(com=="food"){
        b=new Food(comando);
    }else if(com=="feed"||com=="feedid"){
        b=new Feed(comando);
    }else if(com=="nofood"){
        b=new NoFood(comando);
    }else if(com=="empty"){
        b=new Empty(comando);
    }else if(com=="see"){
        b=new See(comando);
    }else if(com=="info"){
        b=new Info(comando);
    }else if(com=="n"){
        b=new N(comando, &xr, &yr,wr,hr, &zonaR, &zona0);
    }else if(com=="anim"){
        b=new Anim(comando);
    }else if(com=="visanim"){
        b=new Visanim(comando,wr,hr,xr,yr);
    }else if(com=="store"){
        b=new Store(comando, &estados);
    }else if(com=="restore"){
        b=new Restore(comando, &estados);
    }else if(com=="load"){
        b=new Load(comando,wr,hr,xr,yr, &estados, &zonaR, &zona0);
    }else if(com=="slide"){
        b=new Slide(comando, &xr, &yr);
    }
    return b;
}
```

Conclusão

Com este trabalho pensamos que conseguimos cumprir todos os requisitos pretendidos, com auxílio do que foi lecionado nas aulas praticas e teóricas, tendo sempre em consideração os princípios da programação orientada a objetos nomeadamente o polimorfismo, a herança e o encapsulamento.